Empório da Música

Technical Project Documentation

Document type: Prototype technical documentation
Application: Text-based customer service agent
Language of the application: Brazilian Portuguese
Data classification: Fictional demonstration data

1. Purpose and scope

Empório da Música is a local customer-service prototype for a fictional musical-instrument store. It provides grounded answers about the catalog, orders, promotions, and store policies through an interactive command-line interface.

The project prioritizes predictable behavior without external services, answers grounded in local data, explicit intent routing, simple installation, and clear fallbacks. It is not a production commerce platform: it does not authenticate customers, connect to a real order-management system, or expose a web API.

2. Functional behavior

Supported intents

Intent	Example	Main source
`GREETING`	`oi`	Fixed persona response
`PRODUCT_INFO`	`qual o preço da guitarra Yamaha FG800?`	Product and promotion CSVs
`ORDER_STATUS`	`qual o status do pedido #1?`	Orders, customers, order-items, and products CSVs
`POLICY`	`aceitam pix?`	Store policy PDF
`OUT_OF_SCOPE`	`vocês vendem cabos?`	Router keyword rules
`UNKNOWN`	An unsupported or ambiguous question	Fixed fallback response

Product responses can return the name, price, description, stock, catalog status, and active promotion. Category and maximum-price filters are also supported, for example `quais violões custam até R$ 1.000?`; matching products are returned in ascending price order. Order responses can return the customer, status, total, tracking code, estimated delivery, and ordered products. Policy responses

search text extracted from `data/políticas_da_loja.pdf`. Accessory keywords take priority over product classification to prevent invented availability results.

3. Technical architecture

```
User
  |
  v
cli.py
  |
  v
agent.py
  |
  +--> intent_router.py ------> Intent + extracted entities
  |
  +--> knowledge_base.py -----> In-memory indexes and grounded lookup
             |
             v
        data_loader.py
           |        |
           v        v
         CSV files  Policy PDF
```

Module responsibilities: `cli.py` handles the terminal; `agent.py` maps intents to handlers; `intent_router.py` classifies and extracts entities; `data_loader.py` reads CSV/PDF sources; `knowledge_base.py` indexes and formats results; `conversation_history.py` persists up to 100 exchanges; and `test_agent.py` provides smoke coverage.

4. Data flow

1. `KnowledgeBase` calls `load_all()` during agent initialization.
2. The loader reads six CSV files and extracts the policy PDF.
3. The knowledge base builds indexes by IDs, names, order items, and promotions.
4. Each message is classified before lookup.
5. The matching handler queries only relevant in-memory data.
6. The agent formats a Portuguese response or a specific fallback.

5. Framework and agent approach

The project uses a deliberately small **hybrid approach**: deterministic intent classification and entity extraction, structured retrieval from CSV files, keyword-based retrieval from the policy PDF, deterministic response templates for reliable facts, and optional local LLM rewriting through Ollama.

This approach was chosen over ReAct, function calling, and a SQL agent because the data set is small and known in advance. Explicit handlers make prices, stock, orders, and policies easier to audit and test.

6. Model and provider

The default provider is the local deterministic pipeline: no model, API key, network connection, or paid service is required. The optional provider is **Ollama**, accessed through its local HTTP endpoint. The default model is `llama3.2:latest`; it can be changed with `--ollama-model` or `OLLAMA_MODEL`. The endpoint defaults to `http://127.0.0.1:11434` and can be changed with `OLLAMA_URL`.

The LLM is not the source of truth. It only rewrites an answer already grounded in local data. If Ollama is unavailable, the model fails, or a factual response is not safely preserved, the deterministic answer is returned.

7. Retrieval architecture and design choices

The implementation uses deterministic retrieval instead of a vector database because the dataset is small, local, and structured. This keeps price, stock, and order responses auditable and avoids arbitrary SQL or tool calls generated by a model. Structured records are indexed in memory by IDs and names. Policy text is extracted from the PDF and searched by topic keywords. Product category and maximum-price filters are applied directly to fields such as `category_id` and `price_brl`.

8. Prompt strategy

When `--ollama` is enabled, the prompt contains the customer's original question, the deterministic answer retrieved from local data, instructions to answer in Brazilian Portuguese, and an instruction not to add prices, addresses, policies, or availability not present in the grounded answer. The model has a wording role rather than a retrieval or database-query role, reducing hallucination risk and keeping local data authoritative.

9. Installation and execution

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python test_agent.py
```

The CSV files and policy PDF must remain in the `data/` directory.

Run without Ollama:

```
python cli.py
```

To enable optional local Ollama response wording, start Ollama and run one of the following commands. The default model is `llama3.2:latest`; use `--ollama-model` to select another installed model. If the local service is unavailable, the deterministic response is retained.

```
python cli.py --ollama
python cli.py --ollama --ollama-model gemma3:latest
```

10. CLI usage

The CLI accepts free-text questions in Brazilian Portuguese and recognizes `/ajuda`, `/help`, `/historico`, `/limpar`, `/status`, `/debug <question>`, `/sair`, `/exit`, and `/quit`. The `--check` option validates that the required CSV tables and policy PDF are loaded without starting the interactive interface.

```
python cli.py --check
```

11. Persistence and privacy considerations

Conversation history is written to `conversation_history.json` and limited to the latest 100 exchanges. Without authentication or user separation, this is suitable only for local demonstrations. Production use would require authenticated access, retention controls, encryption where appropriate, and a transactional store.

12. Technical trade-offs

Deterministic routing instead of an autonomous agent

The data set is small and known in advance, so explicit handlers make answers auditable and reproducible. This reduces hallucinations and avoids arbitrary database queries. The trade-off is less flexibility with ambiguous or unforeseen wording.

Ollama instead of a hosted API

Ollama runs locally without credentials or per-request cost and keeps demonstration data on the local machine. The trade-off is dependence on local hardware and model availability.

CLI instead of a web UI

The challenge explicitly allows a CLI. It keeps the focus on agent behavior and retrieval quality and is easy to reproduce. The trade-off is a less polished end-user experience than a web interface.

JSON history instead of a database

JSON is dependency-free and appropriate for a single-user prototype. The trade-off is no multi-user isolation, transactions, or concurrent-write protection.

13. Testing strategy

`test_agent.py` covers greetings, product, order, policy, out-of-scope, and unknown messages. Future tests should include malformed data, missing files, empty messages, ambiguous intents, unknown IDs, PDF extraction failures, and history write failures.

14. Known limitations and production considerations

The prototype omits authentication, real-time inventory and order integration, a web/API layer, concurrency-safe storage, semantic search, comprehensive automated tests, monitoring, and production-grade error reporting. Ollama response quality also depends on the selected local model and available hardware. These concerns should be addressed before production use.

15. Use of coding assistants

Copilot SDK in VS Code was used as a development assistant. The workflow was to inspect the repository, reproduce behavior, investigate bugs, evaluate implementation options, apply focused changes, review the generated code manually, run tests and syntax checks, and organize the README and technical documentation. Copilot did not replace validation or design decisions; the implementation, tests, local Ollama behavior, and final documentation were checked in the repository environment.

16. Repository map

Path	Responsibility
`agent.py`	Agent orchestration and handlers
`cli.py`	Interactive terminal interface
`data_loader.py`	CSV and PDF loading
`knowledge_base.py`	Normalization and lookups
`intent_router.py`	Intent and entity extraction
`conversation_history.py`	Local persistence
`data/`	Fictional source data
`examples/`	Example conversations

17. Responsible use

All included customer, product, order, and policy information is fictional. Do not use this repository as a source of real customer decisions or as a substitute for authenticated operational systems.